



MalochBot - Technische Dokumentation

Automatisierte Jobsuche, Bewerbungsverwaltung und Karriereunterlagen

Stand: September 2026 - Alexander Riedel - MIT

1. Zweck und Leitidee

MalochBot bündelt die komplette Bewerbungsreise an einem Ort: Stellen finden, Bewerbungen verfolgen, Unterlagen pflegen und Anschreiben erzeugen. Leitidee ist ein **lokaler, nachvollziehbarer Werkzeugsatz**: Jede inhaltliche Analyse läuft über die `opencode`-CLI mit einem frei wählbaren Sprachmodell, und alle Daten liegen in einer mitgelieferten SQLite-Datenbank. Es gibt keine Telemetrie und keine versteckten Netzwerkziele; der gesamte Code ist einsehbar.

Der Name leitet sich vom umgangssprachlichen „malochen“ ab — die Software nimmt die mühsame Routine der Jobsuche ab.

2. Funktionsumfang

- Jobsuche per Knopfdruck: neue, offene Stellen finden, deduplizieren, bewerten.
- Eine konsolidierte Ansicht aller Jobs mit Echtzeit-Filtern und Detailfenster.
- Manuelles Hinzufügen einzelner Stellen.
- Bewerbungs-Tracking: Postfach per IMAP lesen, Status per LLM bewerten.
- Verlauf je Bewerbung inklusive Mailverlauf als Zeitachse.
- Unterlagen-Ablage mit LLM-Bewertung und Erzeugung verbesserter Kopien.
- Anschreiben-Erzeugung auf Basis von Profil, Lebenslauf und Referenzanschriften.
- Anschreiben-Verbesserung per Freitext-Feedback.
- Statistiken mit Diagrammen zu Status, Phasen, Fit, Herkunft und Monatsverlauf.
- Live-Log, Fehlerprotokolle, vier Designs (Hell/Dunkel/Nord/Sepia), Deutsch/Englisch.

3. Architektur

MalochBot ist eine klassische Server-Anwendung mit serverseitig gerenderten Templates.

- **Backend:** Python 3.10+ mit FastAPI und Uvicorn.
- **Datenbank:** SQLite (eine Datei, keine externe Datenbank nötig).
- **Frontend:** Jinja2-Templates sowie handgeschriebenes CSS und JavaScript. Kein Build-Schritt, keine externen CDNs — dadurch offline- und servertauglich.
- **Analyse:** Die opencode-CLI wird als Unterprozess aufgerufen; Ausgaben werden zeilenweise in den Live-Log gestreamt.
- **Hintergrundläufe:** Analysen laufen in Threads; die Oberfläche zeigt den Fortschritt über eine Statusleiste und fragt den Laufzustand über eine kleine JSON-API ab.

Kernmodule unter `app`:

- ``main.py`` — FastAPI-App, Routen, JSON-API, SSE-Stream.
- ``db.py`` — SQLite-Schema, Migrationen, Zugriffshelfer.
- ``secrets.py`` — Secret-Store (Keyring oder verschlüsselte Datei).
- ``logbus.py`` — Live-Logs und Persistenz.
- ``opencode_adapter.py`` — Aufruf der opencode-CLI und robuste JSON-Extraktion.
- ``providers.py`` — Mail-Anbieter-Presets.
- ``i18n.py`` — Übersetzungen (Deutsch/Englisch).
- ``engines/search.py`` — Jobsuche.
- ``engines/tracking.py`` — Postfach-Auswertung.
- ``engines/documents.py`` — Unterlagen und Anschreiben.

4. Datenmodell

Alle fachlichen Daten liegen in SQLite-Tabellen:

- ``jobs`` — jede gefundene Stelle mit Firma, Titel, Ort, Remote, Links, Fit-Score, Begründung, Beschreibung, Status und Anschreiben-Pfad.
- ``applications`` — je Bewerbung die Phase, das Antwortdatum, Kanal und Notizen.
- ``emails`` — ausgewertete Nachrichten mit Zuordnung zu einem Job.
- ``documents`` — hochgeladene Unterlagen mit Art, Version und Bewertungstext.
- ``runs`` — jeder Analyse-Lauf mit Status, Modell und Ergebnis.
- ``logs`` — Zeilen des Live-Logs.
- ``settings`` — Konfiguration ohne Geheimnisse (Modell, Sprache, Schwellen).

Beim Start prüft die Anwendung das Schema und ergänzt fehlende Spalten. Dokumentpfade werden dabei automatisch an den aktuellen Datenordner angepasst, sodass ein Umzug auf einen anderen Rechner ohne Nacharbeit funktioniert.

5. Statuslogik

Der Status eines Jobs wird streng aus Belegen abgeleitet, nicht geraten:

- **Gefunden** — Standard für jede neue Stelle.
- **Vorgemerkt** — manuelle Markierung durch die Nutzerin oder den Nutzer.
- **Beworben** — es liegt eine Eingangsbestätigung oder eine Antwort per Mail vor.
- **Interview / Angebot / Abgelehnt** — aus dem Mailverlauf abgeleitet.
- **Ignoriert** — manuell.

Eine fehlende Rückmeldung stuft einen Job **nicht** auf „Beworben“ hoch. Die

Statuszuordnung ist rangbasiert: Ein Fortschritt wird nie automatisch zurückgesetzt.

6. Die Engines

Suche. Der Prompt an das Modell enthält Profil, Präferenzen, die ausgewählten Unterlagen, bekannte Jobs (zur Deduplizierung) und eine Fit-Schwelle. Das Modell recherchiert über Websuche und liefert eine strukturierte Liste zurück, die in die Datenbank übernommen wird. Bereits bekannte Stellen werden übersprungen.

Tracking. Das Postfach wird read-only und inkrementell per IMAP gelesen (nur neue Nachrichten seit dem letzten Scan). Relevante Mails werden gefiltert und dem Modell zur Klassifikation vorgelegt. Das Ergebnis aktualisiert Status und Bewerbungsdatensätze.

Unterlagen. PDF-, DOCX- und Textdateien werden zu Text extrahiert, per Modell bewertet und auf Wunsch zu einer verbesserten Kopie verarbeitet. Anschreiben werden aus Profil, Lebenslauf und einem Referenzanschreiben erzeugt und als PDF gespeichert.

7. Integration der opencode-CLI

Alle Analysen laufen über ``opencode run -m <modell>``. Die Ausgabe der CLI ist nicht maschinenlesbar garantiert: Sie enthält Statuszeilen, Werkzeugaufrufe und erst am Ende das Ergebnis. Der Adapter parst daher **das letzte gültige JSON-Objekt** aus der Ausgabe statt naiv vom ersten bis zum letzten Klammerzeichen zu lesen — so werden Werkzeugaufrufe wie ``{"query": ...}`` zuverlässig ignoriert. Schlägt die Auswertung fehl, bleibt der letzte gültige Stand erhalten.

Das Modell ist frei wählbar; die Auswahlliste wird zur Laufzeit live aus ``opencode models`` erzeugt, nach Anbieter gruppiert.

8. Oberfläche

Die Startseite ist eine Master-Detail-Ansicht: links die filterbare Liste aller Jobs, rechts das Detailfenster. Die Filter wirken sofort (clientseitig, ohne Neuladen), die Trennung zwischen Liste und Detail ist per Ziehen verstellbar.

Analysen laufen im Hintergrund. Statt Nutzerinnen und Nutzer in ein Log zu schicken, erscheint oben eine **Statusleiste** („Anschreiben wird erzeugt mit LLM: ...“), die bei Erfolg grün wird („Anschreiben für ... wurde erzeugt“). Erst dann erscheint der hervorgehobene Download-Button. Die vollständigen Protokolle bleiben im Reiter „Log“.

9. Sicherheit und Geheimnisse

- Kein Geheimnis im Repository; `.gitignore` schließt den Datenordner aus.
- Zugangsdaten liegen bevorzugt im **Betriebssystem-Keyring**. Ist keiner verfügbar (etwa auf einem Server ohne Desktop), greift eine mit **Fernet verschlüsselte Datei**; der Schlüssel liegt mit Dateirechten 0600 daneben.
- Mailzugriff ist ausschließlich **IMAP read-only** und inkrementell.
- Kein Telemetrie-Versand. Der Code ist vollständig einsehbar.

10. Installation und Dauerbetrieb

Für Linux und Windows liegen Installer bei, die eine virtuelle Umgebung anlegen, die Abhängigkeiten installieren, `opencode` prüfen und die Datenbank initialisieren.

Für den Dauerbetrieb gibt es systemd-Vorlagen (systemweit oder als Benutzerdienst mit Linger). Der Dienst bindet wahlweise nur lokal oder im Netz; für den Zugriff von außen empfiehlt sich ein privates Netz wie Tailscale. Die Datenbank und die Unterlagen lassen sich durch Kopieren des Datenordners migrieren.

11. Logging und Fehlerbehandlung

Jeder Lauf erzeugt Logzeilen in der Datenbank und auf Wunsch in Dateien. Fehler werden mit Kontext protokolliert und in der Oberfläche verständlich angezeigt. Hintergrundläufe sind gegen Abstürze isoliert: Ein Fehler in einer Analyse beendet weder Server noch Oberfläche.

12. Erweiterbarkeit

- **Neue Mail-Anbieter:** ein Eintrag in `providers.py` genügt (Label, Server, Hinweis).
- **Neue Modelle:** erscheinen automatisch, sobald `opencode` sie kennt.
- **Neue Engines:** folgen dem Muster „Prompt → `opencode` → JSON → Datenbank“ und lassen sich als weitere Buttons einbinden.
- **Dokumentarten:** frei definierbar; die Suche nutzt die ausgewählten Dokumente.

13. Grenzen und Ausblick

Sprachmodelle liefern gelegentlich ungenaue Ergebnisse; deshalb ist jeder generative Schritt bewusst als Vorschlag mit anschließender menschlicher Prüfung ausgelegt, und der Anschreiben-Workflow erlaubt direktes Feedback und Korrigieren. Künftige Arbeiten sind unter anderem: eine gestufte automatische Qualitätskontrolle für generierte Inhalte, eine feinere Zuordnung von Mail-Threads und ein optionaler Login für den Mehrbenutzerbetrieb.

14. Lizenz

MIT-Lizenz. Der vollständige Quelltext ist frei einsehbar und nachnutzbar.